

PROPUESTA DE PROYECTO

CHATIFY

Presentado por:

Julian David Albarracin Galindo - jalbarracing@unal.edu.co
Julian David Velandia Neuta - jvelandian@unal.edu.co
Juan David Cruz Giraldo - jcruzgi@unal.edu.co
Emiliano Guerra Gallego - eguerrag@unal.edu.co
Maria Catalina Rodriguez Cardona - marirodriguezcar@unal.edu.co

Profesora:

Liliana Marcela Olarte Mesa

Miércoles 15 de Abril de 2026



Universidad Nacional de Colombia
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas y Computación
2026

Descripción general del sistema

El presente proyecto tiene como objetivo el desarrollo de una aplicación web orientada a la exploración y recomendación musical mediante la integración de servicios externos y el diseño de algoritmos propios. Más allá de replicar funcionalidades existentes en plataformas comerciales, se busca construir un entorno que permita analizar, comprender y experimentar con distintos enfoques de recomendación basados en el comportamiento del usuario.

La motivación principal del proyecto radica en la limitada transparencia de los sistemas de recomendación actuales, los cuales suelen operar como “cajas negras”. En este contexto, la propuesta pretende ofrecer una alternativa donde el proceso de recomendación sea comprensible y controlable, permitiendo evaluar cómo influyen distintos factores, como el historial de reproducción o las preferencias del usuario, en la generación de sugerencias musicales.

Actores

- **Usuario**
Actor que interactúa directamente con la aplicación. Puede explorar contenido musical, realizar búsquedas, reproducir canciones y generar un historial de uso a partir de sus interacciones.
- **Sistema**
Corresponde al actor encargado de procesar la información generada por el usuario y transformarla en sugerencias musicales. Este actor analiza datos como el historial de reproducción, patrones de uso y posibles características de las canciones para identificar preferencias y generar recomendaciones.
- **Servicios externos (Spotify / Last.fm / SoundCloud/ Chatgpt)**
Son plataformas externas que proporcionan datos y/o capacidades esenciales para el funcionamiento del sistema. Dependiendo de la integración, estos servicios pueden ofrecer acceso a catálogos musicales, metadatos de canciones, información sobre artistas o incluso capacidades de reproducción de audio. El sistema se comunica con estas plataformas a través de sus APIs, lo que permite obtener información actualizada sin necesidad de gestionar directamente grandes volúmenes de datos. Adicionalmente, se contempla el uso de modelos de lenguaje como OpenAI (ChatGPT) para la implementación de un módulo basado en RAG. Este permitirá procesar consultas en lenguaje natural realizadas por el usuario, brindar información adicional sobre la canción, artista, álbum o el propio sistema, enriqueciendo la experiencia del usuario.

Funcionalidades

- Autenticación de usuario
- Búsqueda de canciones

- Reproducción de música
- Gestión de playlists
- Registro de historial
- Generación de recomendaciones
- Inclusión de chatbot

Roles en el proyecto

Rol	Nombre	Responsabilidades
Scrum Master	Julian Velandia	Facilita la metodología ágil, coordina reuniones (daily, sprint planning), elimina bloqueos del equipo y asegura que se sigan buenas prácticas de desarrollo.
Backend Developer	Julián Albarracín	Diseña e implementa la API REST, maneja la lógica del sistema, integra servicios externos como Spotify y OpenAI, y gestiona la base de datos y el sistema de recomendación.
Frontend Developer	Catalina Rodriguez	Desarrolla la interfaz de usuario utilizando React y JavaScript, implementa el reproductor de música mediante el SDK de Spotify y gestiona la interacción del usuario con el sistema.
Developer de Lógica y Recomendación	Juan David Cruz	Diseña e implementa los algoritmos de recomendación, analiza el comportamiento del usuario y mejora la personalización del sistema.
DevOps	Emiliano Guerra	Se encarga del despliegue del sistema, configuración de entornos, automatización y monitoreo del funcionamiento de la aplicación.

Nota: Se tiene en cuenta que durante el proyecto se realizará una rotación de los roles entre los integrantes del equipo, con el objetivo de priorizar el conocimiento integral del funcionamiento de la aplicación en cada una de sus partes por todo el equipo.

Tecnologías a usar

Frontend

React (JavaScript) será el framework principal para la construcción de la interfaz de usuario.

Backend

Python con FastAPI será el framework utilizado para construir la API REST del sistema. FastAPI permite definir endpoints con validación automática de datos mediante **Pydantic**, generación de documentación interactiva con **Swagger UI**, y soporte nativo para operaciones asíncronas, lo cual es conveniente para las integraciones con servicios externos. La lógica de negocio, el sistema de recomendación y la comunicación con APIs de terceros serán responsabilidad de este componente.

Base de datos

Se utilizará **PostgreSQL** como motor de base de datos relacional o no relacional (se está estudiando la viabilidad de cada una) para el entorno de producción, de elegirse el modelo relacional será gestionado a través del ORM **SQLAlchemy**.

APIs externas e integraciones

- **Spotify Web API**: proveerá acceso al catálogo musical, metadatos de canciones (título, artista, álbum, género, atributos de audio como *danceability*, *energy*, *tempo*, entre otros).
- **Claude API / OpenAI API**: se integrará para implementar el módulo de chatbot musical. El sistema enviará las consultas del usuario junto con contexto relevante (canción actual, historial reciente) y obtendrá respuestas en lenguaje natural sobre artistas, géneros, álbumes y el funcionamiento del propio sistema.

Sistema de recomendación

El algoritmo de recomendación se implementará directamente en Python utilizando **scikit-learn** y/o **NumPy**. Se calculará la similitud entre canciones basándose en sus atributos de audio (obtenidos desde la Spotify Web API) mediante métricas como la similitud del coseno. Este componente será desarrollado como un módulo independiente dentro del backend, lo cual facilita su iteración y mejora sin afectar el resto del sistema.

Herramientas de desarrollo y DevOps

- **Git / GitHub**: control de versiones y colaboración entre los integrantes del equipo, con uso de ramas por funcionalidad y pull requests para revisión de código.
- **Docker / Docker Compose**: contenerización del backend y la base de datos para garantizar entornos reproducibles entre los distintos equipos de trabajo.
- **Pytest**: framework para pruebas unitarias e integración en el backend.
- **Postman / Swagger UI**: herramientas para documentar y probar los endpoints de la API REST durante el desarrollo.